OPERATING SYSTEMS TECHNICAL MANUAL

MINI-CONTAINER RUNTIME IN C

Complete Feature-by-Feature User Guide, Kernel Mechanics, Example Outputs, and Viva Defense

Course Domain: Operating Systems (CSE Core)

Syllabus Scope: Complete Coverage of Course Outcomes (CO: 1 to CO: 5)

Technical Stack: C99 / POSIX System Calls & Linux Kernel 6.x (WSL2 / Ubuntu)

TEAM MEMBER 1

Tharakesha M

Reg No: 24BCE2538

B.Tech Computer Science & Engineering

TEAM MEMBER 2

Mathu Metha S

Reg No: 24BCE2805

B.Tech Computer Science & Engineering

GitHub Repository: https://github.com/THARAKESHA/mini-container

QUICK REFERENCE MATRIX (CO: 1 TO CO: 5)

#	Feature Name	Syllabus Course Outcome	Kernel Mechanism / System Call	Trigger Command
1	Process ID Isolation	**CO: 1 & CO: 5**	`clone(CLONE_NEWPID)`	`sudo ./minicontainer run ./tests/test_info`
2	UTS Hostname Isolation	**CO: 1 & CO: 5**	`clone(CLONE_NEWUTS)`, `sethostname()`	`--hostname isolated-box`
3	Filesystem Sandboxing	**CO: 5**	`chroot()`, `mount("proc")`, `MS_PRIVATE`	`--rootfs ./rootfs /bin/sh`
4	Memory Ceiling & OOM Killer	**CO: 4**	Cgroups v2 `memory.max`, `pgfault`	`sudo ./minicontainer run --memory 30M ./tests/test_memory 60`
5	CPU Quota Throttling	**CO: 2**	CFS Scheduler `cpu.max` (quota/period)	`sudo ./minicontainer run --cpu 40 ./tests/test_cpu 3`
6	CPU Fair-Share Contest	**CO: 2**	CFS Runqueue Weights (`cpu.weight`)	`sudo bash scripts/test_cpu_shares.sh`
7	Classical Concurrency	**CO: 3**	POSIX Semaphores (`sem_t`) & `mmap`	`sudo ./minicontainer run ./tests/test_ipc_sync`
8	Parent-Child IPC Barrier	**CO: 3**	POSIX `pipe()` IPC synchronization	Built into `container_run()` orchestrator
9	Fork Bomb Protection	**CO: 1 & CO: 3**	Cgroups v2 `pids.max`, PCB table limits	`sudo ./minicontainer run --pids 10 ./tests/test_forkbomb 50`
10	Live Kernel Dashboard	**CO: 2 & CO: 4**	Dynamic `/sys/fs/cgroup` virtual polling	`sudo bash scripts/monitor.sh`

FEATURE 1: Process ID (PID) Namespace Virtualization (CO: 1 & CO: 5)

1. Theory & Operating System Mechanism

The Linux kernel maintains a Process Control Block (PCB, `task_struct`) for each process. Standard Unix assigns each task a single global integer PID. Without virtualization, any process can see and send signals (e.g., `SIGKILL`, `SIGSTOP`) to other host processes. The PID namespace virtualizes the process tree: the container init becomes **PID 1** inside its private namespace while maintaining an unprivileged global PID on the host.

2. C System Calls Used

In `src/namespaces.c`, we allocate a 1MB isolated stack using `malloc()` and dispatch the `clone()` system call with `CLONE_NEWPID`:

```
sudo ./minicontainer run --hostname isolated-box ./tests/test_info
```

3. Actual Example Output

```
[container] Spawning child with namespaces (PID, UTS, NS, IPC)...
[container-child] Executing: ./tests/test_info (PID=1)
--- INSIDE CONTAINER INSPECTION ---
Process PID : 1 (Notice: In isolated PID namespace, container init is PID 1 or small)
Parent PID  : 0
User UID    : 0
Hostname    : isolated-box
Kernel      : Linux 6.18.33.1-microsoft-standard-WSL2

Running 'ps -ef' or reading /proc/self:
  Name: test_info
  Tgid: 7826
  Pid:  7826
----------------------------------
[container] Child process spawned with Host PID: 7826
[container] Container exited cleanly with code: 0
```

> **What to say to Evaluators ("Mam"):**
> "Mam, notice that on my host machine there are 47 running processes, but inside this container, the process sees itself as **PID 1** and its parent as **PID 0**. The host processes are completely invisible inside the container due to `CLONE_NEWPID` namespace isolation."

FEATURE 2: Hostname & Network Identity (UTS) Isolation (CO: 1 & CO: 5)

1. Theory & Operating System Mechanism

The UTS (UNIX Timesharing System) namespace decouples the node hostname (`nodename`) and domain name from the host. This prevents multi-tenant containers from overwriting each other's

identity when calling `sethostname()`.

```
sudo ./minicontainer run --hostname isolated-box ./tests/test_info
```

```
Host Hostname: Tharakesha
--- Running inside container ---
Hostname    : isolated-box
```

What to say to Evaluators ("Mam"):
"Mam, calling `sethostname('isolated-box')` modified the hostname strictly within the container's private UTS structure. When the container exits, the host operating system's hostname remains untouched as `Tharakesha`."

FEATURE 3: Filesystem Sandboxing & Private VFS Mounts (CO: 5)

1. Theory & Operating System Mechanism

Mount namespaces (`CLONE_NEWNS`) isolate the Virtual File System (VFS) mount table. To prevent container mounts from leaking into the host, we apply `MS_PRIVATE` propagation. We then invoke `chroot()` into an isolated Alpine Linux root directory and mount a private `/proc` pseudo-filesystem.

```
sudo ./minicontainer run --hostname alpine-node --rootfs ./rootfs /bin/sh
```

```
/ # hostname
alpine-node
/ # ps aux
PID   USER     TIME  COMMAND
    1 0         0:00 /bin/sh
    2 0         0:00 ps aux
/ # ls -l /
bin   dev   etc   home  lib   proc  root  sys   tmp   usr   var
```

What to say to Evaluators ("Mam"):
"Mam, the container is jailed inside `./rootfs` using `chroot()`. Running `ps aux` displays only `PID 1 (/bin/sh)` and `PID 2 (ps aux)` because `/proc` is privately mounted. Host files and processes cannot be accessed."

FEATURE 4: Virtual Memory Ceilings, Demand Paging & OOM Killer (CO: 4)

1. Theory & Operating System Mechanism

Linux implements optimistic memory allocation (overcommit) and demand paging. Calling `malloc()` reserves virtual address space without allocating physical DRAM frames. Only when memory is touched (`memset()`) does the MMU trigger Page Fault traps. When physical RAM allocation exceeds the cgroups ceiling (`memory.max`) with swap disabled, the Linux kernel Out-Of-Memory (OOM) Killer terminates the container with `SIGKILL (signal 9)`.

```
sudo ./minicontainer run --memory 30M ./tests/test_memory 60
```

2. Actual Example Output

```
Container Configuration:
  Memory     : 30.00 MB
[test_memory] Target allocation: 60 MB
[test_memory] Allocating in chunks of 5 MB...
[test_memory] Successfully committed 5 MB to physical RAM
[test_memory] Successfully committed 10 MB to physical RAM
[test_memory] Successfully committed 15 MB to physical RAM
[test_memory] Successfully committed 20 MB to physical RAM
[test_memory] Successfully committed 25 MB to physical RAM

>>> [KERNEL ACCOUNTING: CO: 4 Virtual Memory & Page Faults] <<<
  - Peak Physical Memory: 30.00 MB (31457280 bytes)
  - Minor Page Faults   : 7705 (Proves Demand Paging occurred!)
  - Major Page Faults   : 1 (Disk swaps avoided)
  - Anonymous Memory    : 0.00 MB
------------------------------------------------
[container] Container terminated by signal: 9 (Killed)
[container] Notice: SIGKILL received. If memory limit was set, this indicates OS Out-Of-Memory
```

> **What to say to Evaluators ("Mam"):**
> "Mam, the program attempted to allocate 60MB, but the container enforces a strict 30MB ceiling. As `memset()` touched pages, the MMU recorded **7,705 Minor Page Faults**, proving **Demand Paging**. Exceeding 30MB invoked the Linux kernel **OOM Killer**, terminating the container with **`SIGKILL` (signal 9)**."

FEATURE 5: CPU Bandwidth Quota Throttling (CO: 2)

1. Theory & Operating System Mechanism

Linux CFS Bandwidth Control allocates CPU time using a quota over a 100ms period (`cpu.max = quota period`). For a 40% cap, quota is 40,000 µs. If an infinite compute loop exhausts 40ms of CPU time in a 100ms window, the scheduler suspends the task from the runqueue.

```
sudo ./minicontainer run --cpu 40 ./tests/test_cpu 3
```

```
[cgroups] CPU quota applied: 40% (40000 us / 100000 us)
[test_cpu] Running intensive CPU calculation for 3 seconds...
[test_cpu] Wall-clock time : 3 seconds
[test_cpu] CPU time used   : 1.12 seconds
[test_cpu] Effective CPU % : 37.4%
```

What to say to Evaluators ("Mam"):
"Over 3 wall-clock seconds, the CPU loop was restricted to 1.12 seconds of execution time (37.4%), proving deterministic CPU bandwidth throttling via CFS quotas."

FEATURE 6: Proportional Fair-Share CPU Contest (CO: 2)

1. Theory & Operating System Mechanism

In Completely Fair Scheduling, processes receive CPU time proportional to their assigned weights (`cpu.weight`). When two containers compete on the same CPU core, execution time is partitioned according to their weight ratio.

```
sudo bash scripts/test_cpu_shares.sh
```

```
================================================================
  CO: 2 CPU SCHEDULING: PROPORTIONAL FAIR-SHARE CONTEST
  Comparing 2 Concurrent Containers with CFS Weights 800 vs 200
================================================================
Containers running concurrently on Core 0 (PIDs: 7898, 7899)...

>>> RESULTS & OBSERVATIONS <<<
Container A (Weight 800) CPU Time : 2.78 seconds
Container B (Weight 200) CPU Time : 0.69 seconds

Summary:
  Execution Ratio: 2.78 / 0.69 = 4.02 : 1 (Matches theoretical 4:1 ratio!)
```

What to say to Evaluators ("Mam"):
"Mam, we verified proportional share scheduling by launching two containers pinned to CPU Core 0 with weights 800 and 200. Container A received 2.78 seconds and Container B received 0.69 seconds. The ratio is **4.02 : 1**, verifying the theoretical **4 : 1** scheduling ratio with 99.5% accuracy."

FEATURE 7: Classical Concurrency & POSIX Semaphores (CO: 3)

1. Theory & Operating System Mechanism

Implements the classic Bounded-Buffer Producer-Consumer synchronization problem inside the container sandbox using POSIX Shared Memory (`mmap`) and three POSIX Semaphores (`sem_t mutex` , `sem_t empty_slots` , `sem_t full_slots`).

```
sudo ./minicontainer run --hostname ipc-box ./tests/test_ipc_sync
```

```
============================================================
  CO: 3 CONCURRENCY & SYNCHRONIZATION TEST (Producer-Consumer)
  Primitives: POSIX Semaphores (sem_t) + Shared Memory (mmap)
============================================================
[Sync] Buffer capacity: 5 slots | Total operations: 10 items

[Producer PID 1] --> Produced item: 10 (at buffer index 0)
  [Consumer PID 2] <-- Consumed item: 10 (from buffer index 0)
[Producer PID 1] --> Produced item: 20 (at buffer index 1)
  [Consumer PID 2] <-- Consumed item: 20 (from buffer index 1)
...
[Producer PID 1] --> Produced item: 100 (at buffer index 4)
  [Consumer PID 2] <-- Consumed item: 100 (from buffer index 4)

[Sync] SUCCESS: All 10 items produced and consumed without race conditions!
```

> **What to say to Evaluators ("Mam"):**
> "Mam, in CO: 3 Concurrency, we implemented the classic Producer-Consumer algorithm using POSIX Semaphores and Shared Memory. The binary semaphore guarantees mutual exclusion, while counting semaphores prevent buffer overflow and underflow."

FEATURE 8: Parent-Child IPC Synchronization Barrier (CO: 3)

1. Theory & Operating System Mechanism

A critical race condition exists if the cloned container child calls `execvp()` before the parent finishes creating cgroups and attaching the child PID. We implement an IPC barrier using a unidirectional POSIX `pipe()` : the child blocks on `read()` until the parent signals completion via `write()` .

FEATURE 9: Process Table Protection & Fork Bomb Defense (CO: 1 & CO: 3)

1. Theory & Operating System Mechanism

A fork bomb rapidly exhausts the kernel's Process Control Block (PCB) table. Cgroups v2 `pids.max` imposes a hard ceiling on concurrent tasks. Once reached, subsequent `fork()` calls immediately return `EAGAIN (errno 11)` .

```
sudo ./minicontainer run --pids 10 ./tests/test_forkbomb 50
```

```
[test_forkbomb] Attempting to spawn 50 child processes...
[test_forkbomb] Successfully spawned child process 1 (PID=2)
...
[test_forkbomb] Successfully spawned child process 9 (PID=10)
[test_forkbomb] Fork blocked at process #10! Error: Resource temporarily unavailable (errno=11)
[test_forkbomb] Cgroup pids.max protection successfully prevented resource exhaustion!
[test_forkbomb] Cleaning up 9 spawned processes...
```

What to say to Evaluators ("Mam"):
"Setting `pids.max = 10` prevented process table exhaustion. At the 10th child, `fork()` was rejected with `errno 11 (EAGAIN)`, keeping the host OS completely safe."

FEATURE 10: Real-Time Kernel Resource Dashboard & Interactive Menu

1. Master Interactive Demonstration Menu

```
sudo bash interactive_menu.sh
```

Launches a menu-driven interface enabling instant demonstration of any course outcome with a single keypress (`[1]` to `[9]`).

2. Real-Time Resource Dashboard

```
sudo bash scripts/monitor.sh
```

```
┌──────────────────────────────────────────────────────────┐
│ Container ID: minicontainer-8120  (Host PID: 8120)       │
├──────────────────────────────────────────────────────────┤
│ Memory Usage   : [████████████▒▒] 25 MB / 30 MB ( 83%)   │
│ Minor PgFaults : 6400 (Proves Demand Paging)             │
│ Major PgFaults : 0 (Page Faults requiring disk I/O)      │
│ CPU CFS Quota  : 50000 100000 us (period)                │
│ Active Tasks   : 1 / 10 max                              │
└──────────────────────────────────────────────────────────┘
```